

Part 1:

task1:

1: SELECT * FROM hero WHERE age >= 18 ORDER BY age DESC;

```
app_db=# select * from hero where age >= 18 order by age asc;
 id |  name  | age | team_id
----+-----+----+-----
  2 | Natasha |  35 |      1
  1 | Tony   |  45 |      1
  3 | Logan  | 150 |      2
(3 rows)
```

2:

```
app_db=# SELECT hero.name, team.name AS team_name
app_db=# FROM hero
app_db=# LEFT JOIN team ON hero.team_id = team.id;
 name | team_name
-----+-----
 Tony | Avengers
Tony | Avengers
 Logan | X-Men
Peter |
(4 rows)
```

3:

```
app_db=# SELECT team.name, COUNT(hero.id)
app_db=# FROM team
app_db=# LEFT JOIN hero ON team.id = hero.team_id
app_db=# GROUP BY team.name;
 name | count
-----+-----
X-Men |      1
Avengers |      2
(2 rows)
```

4:

```
app_db=# select * from hero
app_db=# order by id
app_db=# limit 2 offset 2;
 id |  name  | age | team_id
----+-----+----+-----
  3 | Logan  | 150 |      2
  4 | Peter  |  16 |
(2 rows)
```

Question 1:

```

app_db=# INSERT INTO team (name, headquarters) VALUES ('Avengers', 'Los Angeles');
ERROR: duplicate key value violates unique constraint "team_name_key"
DETAIL: Key (name)=(Avengers) already exists.
app_db=# INSERT INTO hero (name, team_id) VALUES ('Ghost', 99);
ERROR: insert or update on table "hero" violates foreign key constraint "hero_team_id_fkey"
DETAIL: Key (team_id)=(99) is not present in table "team".
app_db=# INSERT INTO hero (age) VALUES (30);
ERROR: null value in column "name" of relation "hero" violates not-null constraint
DETAIL: Failing row contains (6, null, 30, null).
app_db=# DELETE FROM team WHERE id = 1;
ERROR: update or delete on table "team" violates foreign key constraint "hero_team_id_fkey" on table "hero"
DETAIL: Key (id)=(1) is still referenced from table "hero".

```

Lệnh 1 (INSERT INTO team... 'Avengers'): Bị chặn bởi ràng buộc UNIQUE. Lỗi báo duplicate key value vì tên 'Avengers' đã tồn tại trong bảng team, mà cột name được quy định là không được trùng lặp (UNIQUE).

Lệnh 2 (INSERT INTO hero... team_id 99): Bị chặn bởi ràng buộc FOREIGN KEY (khóa ngoại). Lỗi báo Key (team_id)=(99) is not present in table "team" vì bạn đang cố gán hero vào một team có ID là 99, nhưng ID này không hề tồn tại trong bảng team.

Lệnh 3 (INSERT INTO hero (age) VALUES (30)): Bị chặn bởi ràng buộc NOT NULL. Lỗi báo null value in column "name" ... violates not-null constraint vì lúc tạo bảng, cột name được yêu cầu bắt buộc phải có dữ liệu (NOT NULL), nhưng lệnh chèn của bạn lại bỏ trống cột này.

Lệnh 4 (DELETE FROM team WHERE id = 1): Bị chặn bởi ràng buộc FOREIGN KEY. Lỗi báo Key (id)=(1) is still referenced from table "hero". Bạn không thể xóa team Avengers (ID = 1) vì trong CSDL vẫn đang có các anh hùng (Tony, Natasha) tham chiếu đến team này. Cơ sở dữ liệu ngăn chặn điều này để tránh việc các anh hùng bị "mồ côi" tham chiếu.

Question 2:

Trong quan hệ 1-N, một team có thể có nhiều hero, nhưng mỗi hero chỉ thuộc về đúng một team. Đặt cột team_id (khóa ngoại) ở bảng hero cho phép mỗi dòng hero lưu trữ đúng một giá trị tham chiếu đến team của mình. Nếu đặt ở bảng team, chúng ta không thể lưu một danh sách (mảng) gồm nhiều ID của các hero vào trong cùng một ô dữ liệu của CSDL quan hệ một cách chuẩn mực được.

Question 3:

Để giải quyết quan hệ "Nhiều-Nhiều" (many-to-many), ta không thể chỉ dùng 2 bảng mà bắt buộc phải có một bảng trung gian (link table). Cấu trúc cần thiết bao gồm:

Bảng hero: Có khóa chính (PK) là id.

Bảng mission: Có khóa chính (PK) là id.

Bảng hero_mission (bảng trung gian): Cần ít nhất 2 cột là hero_id (Khóa ngoại trỏ đến hero.id) và mission_id (Khóa ngoại trỏ đến mission.id). Khóa chính của bảng này sẽ là khóa phức hợp (Composite Primary Key) kết hợp từ cả hai cột này để đảm bảo một hero không bị gán trùng vào cùng một mission hai lần.

Part 2:

Question 4:

Bảo mật: Tránh việc vô tình đẩy thông tin đăng nhập (mật khẩu, tên user) lên các kho lưu trữ mã nguồn (như GitHub), giúp bảo vệ cơ sở dữ liệu khỏi kẻ gian.

Tính linh hoạt: Giúp dễ dàng triển khai ứng dụng trên nhiều môi trường khác nhau (môi trường phát triển cục bộ, môi trường kiểm thử, môi trường thực tế) bằng cách chỉ cần thay đổi biến môi trường trên server mà không cần phải sửa bất kỳ dòng code nào.

Part 3:

Question 5:

Vì trước khi một đối tượng (object) được lưu vào cơ sở dữ liệu (tức là trước khi gọi lệnh commit), nó chưa hề có ID. Chính cơ sở dữ liệu (PostgreSQL) mới là nơi tự động sinh ra con số ID này. Khai báo None giúp Python hiểu rằng việc đối tượng chưa có ID lúc mới tạo ra trong bộ nhớ là hợp lệ.

Question 6:

Các thuộc tính trở thành cột: id, name, age, team_id, secret_name.

Thuộc tính KHÔNG trở thành cột: team (vì nó được định nghĩa bằng Relationship).

Tác dụng của back_populates: Nó báo cho SQLAlchemy biết rằng thuộc tính team bên class Hero và thuộc tính heroes bên class Team là hai đầu của cùng một mối quan hệ. Khi bạn cập nhật một đầu, đầu kia sẽ tự động nhận biết.

Part 4:

```
(.venv) PS C:\Users\Admin\hero-api> fastapi dev app/main.py

⚡ Starting FastAPI in development mode
📄 Using import string: app.main:app
🌐 Server started at http://127.0.0.1:8000
   Documentation at http://127.0.0.1:8000/docs

Logs:
  Will watch for changes in these directories: ['C:\Users\Admin\hero-api']
  Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
  Started reloader process [13160] using WatchFiles
  Started server process [23532]
  Waiting for application startup.
2026-10-05 22:20:19,989 INFO sqlalchemy.engine.Engine BEGIN (implicit)
```

Question 7:

SQLAlchemy tạo ra bảng chi tiết và chặt chẽ hơn. Nó thêm tự động các ràng buộc như NOT NULL ở các trường bắt buộc (name, secret_name). Nó cũng tự động sinh ra các câu lệnh CREATE INDEX cho các trường name và age (vì ở Part 3 ta đặt index=True), giúp tối ưu hóa việc tìm kiếm mà ở Part 1 chúng ta không tự viết.

Question 8:

Khi chạy lại, lệnh CREATE TABLE sẽ KHÔNG in ra nữa. Hàm create_all sinh ra lệnh SQL có chứa mệnh đề ngăn định kiểm tra sự tồn tại (IF NOT EXISTS). Nếu nó kiểm tra thấy bảng hero và team đã có trong PostgreSQL rồi, nó sẽ an toàn bỏ qua và không làm gì cả, giúp dữ liệu không bị xóa mất.

Question 9:

Đó là dòng from app.models import Hero, Team. Nếu thiếu dòng này, SQLAlchemy.metadata sẽ trống rỗng và sẽ không có bảng nào được tạo ra.

Part 5:

Question 10:

Nếu thiếu commit(), API có thể trả về lỗi 500 hoặc trả về JSON thiếu ID (ID = null), và chắc chắn dòng dữ liệu đó chưa được lưu vào PostgreSQL (lệnh SELECT * FROM hero; sẽ không thấy). Hàm add() chỉ có tác dụng "đưa vào hàng đợi" (đưa object vào theo dõi trong session bộ nhớ tạm). Hàm refresh() dùng để nạp lại dữ liệu mới nhất từ CSDL vào object (ví dụ: lấy cái ID vừa được CSDL tự động tạo ra để gán vào object Python).

Question 11:

Nhờ bật echo=True, bạn sẽ thấy Terminal in ra câu lệnh: UPDATE hero SET age=17 WHERE hero.id = ?. Nó chỉ cập nhật cột age. Lý do là vì ta đã dùng hàm exclude_unset=True, hàm này sẽ loại bỏ tất cả các trường bị bỏ trống (None), chỉ giữ lại đúng những trường mà client cố tình gửi lên để cập nhật.

Question 12:

Không có trường secret_name trong JSON trả về. Dòng code chịu trách nhiệm bảo mật thông tin này chính là response_model=HeroPublic nằm bên trong decorator @app.get(...). Nó bắt ép FastAPI chỉ được lọc và trả về các trường có định nghĩa trong class HeroPublic (vốn không hề có secret_name).

```
(.venv) PS C:\Users\Admin\Documents\fastapi dev app/main.pyp trình web\hero-api>

⚡ Starting FastAPI in development mode

🔗 Using import string: app.main:app

🌐 Server started at http://127.0.0.1:8000
Documentation at
http://127.0.0.1:8000/docs

Logs:

| Will watch for changes in these
| directories:
| ['C:\Users\Admin\Downloads\Thiết kế
| và lập trình web\hero-api']
| Uvicorn running on http://127.0.0.1:8000
| (Press CTRL+C to quit)
| Started reloader process [2076] using
| WatchFiles
| Started server process [28400]
| Waiting for application startup.
2026-10-03 09:41:33,042 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-10-03 09:41:33,042 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-10-03 09:41:33,044 INFO sqlalchemy.engine.Engine select current_schema()
2026-10-03 09:41:33,044 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-10-03 09:41:33,045 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-10-03 09:41:33,045 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-10-03 09:41:33,049 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-10-03 09:41:33,052 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_catalog.pg_class.relnamespace
```

Part 6:

Question 13:

Các giá trị như 18 hay biến chuỗi không được nối trực tiếp vào câu SQL. Thay vào đó, chúng được hiển thị dưới dạng các biến liên kết (bind parameters / placeholders) (ví dụ: WHERE hero.age = \$1 hoặc %(age)s). Cách này ngăn chặn triệt để lỗ hổng bảo mật **SQL Injection**, vì CSDL sẽ phân tách rõ ràng giữa mã lệnh SQL và dữ liệu thực thi, không cho phép dữ liệu do người dùng nhập vào được biên dịch thành mã lệnh.

Question 14:

Lọc dữ liệu bằng .where() đẩy công việc tính toán xuống cho cơ sở dữ liệu thực hiện. Cơ sở dữ liệu được tối ưu hóa cực tốt cho việc này (đặc biệt khi có Index). Nếu tải toàn bộ bảng (có thể chứa hàng triệu dòng) lên RAM bằng Python rồi mới lọc, ứng dụng sẽ bị cạn kiệt bộ nhớ (Out of Memory), tốc độ xử lý cực kỳ chậm và gây lãng phí băng thông mạng giữa ứng dụng và CSDL.

Part 7:

Question 15:

Hàm SQLAlchemyModel.metadata.create_all(engine) hoạt động theo cơ chế chỉ khởi tạo các bảng chưa tồn tại. Do bảng hero ở Part 4 đã có sẵn dưới cơ sở dữ liệu, lệnh này sẽ tự động bỏ qua. Sang Part 7, vì mission và heromissionlink là hai model mới hoàn toàn chưa có cấu trúc bảng tương ứng, hệ thống buộc phải thực thi lệnh CREATE TABLE để thiết lập chúng.

Execute

Create

Responses

Curl

```
curl -X 'POST' \
  'http://127.0.0.1:8000/heroes/' \
  -H 'accept: */*' \
  -H 'Content-Type: application/json' \
  -d '{
    "name": "Spider-Man",
    "age": 20,
    "team_id": null,
    "secret_name": "Peter"
  }'
```

Request URL

http://127.0.0.1:8000/heroes/

Server response

Code	Details
200	Response body <pre>{ "name": "Spider-Man", "age": 20, "team_id": null, "id": 1 }</pre> Response headers <pre>content-length: 52 content-type: application/json date: Sat,03 Oct 2026 03:49:53 GMT server: uvicorn</pre>

Responses

Code	Description	Links
------	-------------	-------

```
(.venv) PS C:\Users\Admin\Downloads\Thiết kế và lập trình web\hero-api> psql "postgres://app:secret@localhost:5432/appdb" -c "SELECT
T * FROM heromissionlink;"
 hero_id | mission_id
-----+-----
      3 |          1
(1 row)
```

Part 8:

Question 16:

SQLAlchemy xử lý việc thêm dữ liệu (INSERT) rất thông minh bằng cách ưu tiên các bảng không phụ thuộc. Nó sẽ tạo dữ liệu cho team và mission trước, rồi mới đến hero, và chốt lại bằng bảng trung gian heromissionlink. Nhờ thiết lập relationships, hệ thống sẽ tự động lưu avengers để lấy ID (ví dụ id=1), dùng chính ID này điền vào trường khóa ngoại team_id của anh hùng tương ứng, rồi mới đẩy dữ liệu của anh hùng đó vào database.

Responses

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/heroes/1/missions' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/heroes/1/missions
```

Server response

Code	Details
200	Response body <pre>[{ "title": "Battle of Sokovia", "id": 1 }, { "title": "Infinity War", "id": 2 }]</pre> Response headers <pre>content-length: 70 content-type: application/json date: Sat, 03 Oct 2026 04:13:03 GMT server: uvicorn</pre>

Curl

```
curl -X 'GET' \
'http://127.0.0.1:8000/teams/1/heroes' \
-H 'accept: */*'
```

Request URL

```
http://127.0.0.1:8000/teams/1/heroes
```

Server response

Code	Details
200	Response body <pre>[{ "name": "Iron Man", "age": 45, "team_id": 1, "id": 1 }, { "name": "Captain America", "age": 100, "team_id": 1, "id": 2 }, { "name": "Thor", "age": 1500, "team_id": 1, "id": 3 }]</pre> Response headers <pre>content-length: 151 content-type: application/json date: Sat, 03 Oct 2026 04:12:53 GMT server: uvicorn</pre>

Question 17:

Cột power hiện chưa có mặt dưới database (thể hiện qua lệnh `\d hero`), nên khi bạn gọi API `GET /heroes`, Python sẽ văng lỗi 500 do cố gắng liên kết với một trường không tồn tại. Để giải quyết tình trạng lệch cấu trúc này, tuyệt đối không được dùng cách "xóa sạch bảng rồi tạo lại" nếu hệ thống đang chạy thật (production), vì điều đó đồng nghĩa với việc xóa vĩnh viễn toàn bộ dữ liệu quan trọng của người dùng.

Question 18:

def upgrade() -> None:

```
"""Upgrade schema."""

# ### commands auto generated by Alembic - please adjust! ###

pass

# ### end Alembic commands ###
```

def downgrade() -> None:

```
"""Downgrade schema."""

# ### commands auto generated by Alembic - please adjust! ###

Pass

# ### end Alembic commands ###
```

Question 19:

Bảng alembic_version (có thể thấy khi gõ \dt) được Alembic tạo ra riêng để ghi nhớ hệ thống đang ở phiên bản (revision) nào. Chính vì vậy, bạn phải đưa thư mục migrations/ lên Git. Điều này giúp các thành viên khác trong team và server chạy thật đều nắm được toàn bộ lịch sử thay đổi, đảm bảo việc cập nhật database luôn diễn ra đồng bộ, không bị lệch cấu trúc.

Question 20:

Thay vì đổi tên, hệ thống lại tự động tạo mã lệnh xóa hẳn cột secret_name rồi mới thêm cột alias. Quá trình này cực kỳ nguy hiểm vì dữ liệu của các siêu anh hùng sẽ bốc hơi vĩnh viễn. Cách xử lý an toàn nhất là mở file code vừa được sinh ra và tự gõ lại thành lệnh: op.alter_column('hero', 'secret_name', new_column_name='alias').

```
(.venv) PS C:\Users\Admin\Downloads\Thiết kế và lập trình web\hero-api> alembic history
<base> -> b3b200f4d264 (head), add hero.power
```

```
(.venv) PS C:\Users\Admin\Downloads\Thiết kế và lập trình web\hero-api> alembic current
INFO [alembic.runtime.migration] Context impl PostgresqlImpl.
INFO [alembic.runtime.migration] Will assume transactional DDL.
b3b200f4d264 (head)
```

What is a primary key for? (Khóa chính dùng để làm gì?)

Dùng để định danh duy nhất (uniquely identify) mỗi dòng bản ghi trong một bảng cơ sở dữ liệu.

What is the main benefit of an ORM? (Lợi ích chính của ORM là gì?)

Cho phép lập trình viên tương tác với cơ sở dữ liệu thông qua các object (đối tượng) và code Python quen thuộc, thay vì phải viết các câu lệnh truy vấn SQL thô.

What does `table=True` change about a `SQLModel` class? (`table=True` làm thay đổi điều gì ở một class `SQLModel`?)

Nó thông báo cho `SQLModel` biết rằng class này đại diện cho một **bảng thực sự** dưới cơ sở dữ liệu, chứ không chỉ là một data model dùng để kiểm tra tính hợp lệ của dữ liệu (như Pydantic schema thông thường).

After `session.add(...)`, what actually saves the data to the database? (Sau `session.add(...)`, lệnh nào thực sự lưu dữ liệu xuống database?)

Lệnh **`session.commit()`**.

What does `session.exec(select(Model)).all()` return? (Lệnh này trả về cái gì?)

Trả về một **danh sách (list) Python** chứa tất cả các đối tượng (instances) của Model đó có trong cơ sở dữ liệu.

What is Alembic for? (Alembic dùng để làm gì?)

Là công cụ Migration dùng để quản lý, theo dõi và áp dụng các thay đổi về mặt cấu trúc (schema) của cơ sở dữ liệu (ví dụ: thêm/xóa cột, đổi tên) một cách có hệ thống mà không làm mất dữ liệu hiện có.

You add a field to a model and restart the app. Why is there no new column? (Bạn thêm một trường vào model và khởi động lại app. Tại sao không có cột mới xuất hiện?)

Vì lệnh `SQLModel.metadata.create_all()` chạy lúc khởi động app có quy tắc là **chỉ tạo những bảng chưa tồn tại**. Nếu bảng đã có sẵn, nó sẽ bỏ qua hoàn toàn và không tự động cập nhật thêm cột mới. Bạn phải dùng Alembic để thực hiện việc cập nhật cột này.